

MACHINE LEARNING PROJECT

DOCUMENTATION

"Sistem Validasi Kelayakan Sampah dengan Pendekatan YOLOv8 untuk Object Detection dan Random Forest Ensemble untuk Klasifikasi"



Disusun Oleh:

11423003	:	Samuel Rizki Sinambela
11423010	:	Agatha C. S. Siahaan
11423025	:	Yos Ivan Sirait
11423028	:	Grace Advryanti Tampubolon
11423054	:	Natan Obet Nego Hutahaeen

PROGRAM STUDI TEKNOLOGI REKAYASA PERANGKAT LUNAK

**FAKULTAS VOKASI
INSTITUT TEKNOLOGI DEL
2026**

BAB I.....	4
PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Rumusan Masalah.....	4
1.3 Tujuan.....	4
1.4 Alasan Pemilihan Topik dan Algoritma.....	4
BAB II.....	6
PENGUMPULAN DAN PREPROCESSING DATA.....	6
2.1 Pengumpulan Data (Data Collection).....	6
2.1.1 Sumber Dataset.....	6
2.1.2 Struktur Dataset.....	6
2.1.3 Cara Pengambilan Dataset.....	6
2.1.4 Konteks dan Karakteristik Data.....	7
2.2 Pra-Pemrosesan Data (Data Preprocessing).....	7
2.2.1 Penanganan Missing Value (Orphan Files).....	7
2.2.2 Penanganan Noise dan Outlier.....	8
2.2.3 Validasi Label YOLO (Deteksi Outlier Bounding Box).....	8
2.2.4 Pengecekan Gambar Korup.....	8
2.2.5 Analisis Class Imbalance dan Distribusi Bounding Box.....	9
2.3 Data Transformation.....	10
2.3.1 Feature Engineering.....	10
2.3.2 Encoding.....	11
2.3.3 Scaling dan Normalisasi.....	11
BAB III.....	12
IMPLEMENTASI DAN TRANSFORMASI MODEL.....	12
3.1 YOLOv8 (Object Detection).....	12
3.1.1 Pengertian YOLOv8.....	12
3.1.2 Arsitektur YOLOv8.....	12
3.1.3 Implementasi YOLOv8.....	12
3.1.4 Output Model.....	13
3.1.4 Ekstraksi Fitur dari Hasil Deteksi YOLO.....	14
3.2 Klasifikasi.....	16
3.2.1 Pengertian Klasifikasi.....	16
3.2.2 Algoritma Klasifikasi.....	16
3.2.3 Implementasi Klasifikasi.....	16
3.2.4 Training Model Random Forest.....	17
3.2.5 Evaluasi Model Random Forest.....	18
3.2.6 Hyperparameter Tuning.....	19
BAB IV.....	21
HASIL DAN PEMBAHASAN.....	21
4.1 Evaluasi Model YOLOv8.....	21

4.2 Evaluasi Model Klasifikasi Random Forest.....	21
4.3 Analisis Kritis Hasil Model (Feature Importance & Error Analysis).....	21
4.3.1 Feature Importance.....	21
4.3.2 Analisis Galat (Error Analysis).....	21
BAB V.....	22
PIPELINE INREFENSI END-TO-END.....	22
5.1 Alur Lengkap Pipeline.....	22
5.2 Penyimpanan dan Distribusi Model.....	22
BAB VI.....	24
INSIGHT DAN KESIMPULAN.....	24
6.1 Insight.....	24
6.1.1 Kelebihan Pendekatan Hybrid Yolo dan Random Forest.....	24
6.1.1 Keterbatasan Pendekatan Hybrid Yolo dan Random Forest.....	24
6.2 Kesimpulan.....	25
6.3 Rekomendasi.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan sampah merupakan tantangan krusial pada domain lingkungan. Salah satu titik kritis dalam sistem ini adalah manajemen pengangkutan. Seringkali armada truk sampah mendatangi lokasi penampungan yang volumenya belum cukup untuk diangkut, sehingga menyebabkan pemborosan biaya operasional dan bahan bakar. Sebaliknya, keterlambatan pengangkutan memicu masalah sanitasi. Untuk mengatasi masalah ini, proyek ini membangun Trash Validation System, sebuah sistem cerdas berbasis Machine Learning yang memvalidasi kondisi visual tumpukan sampah secara otomatis untuk menghasilkan keputusan biner: "Tidak Layak" atau "Layak Diangkut".

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam proyek ini adalah:

1. Bagaimana melatih model deteksi objek yang akurat untuk mengenali sebaran sampah dalam berbagai variasi gambar?
2. Bagaimana mengekstrak informasi visual hasil deteksi menjadi fitur data numerik (tabular) yang bermakna?
3. Bagaimana menerapkan hyperparameter tuning dan menggabungkan berbagai algoritma klasifikasi (Ensemble) untuk menentukan kelayakan angkut?
4. Bagaimana membangun pipeline end-to-end yang mengintegrasikan deteksi visi dan klasifikasi menjadi satu kesatuan?

1.3 Tujuan

Tujuan utama pemodelan ini mencakup:

1. Mendeteksi objek sampah dalam gambar secara otomatis menggunakan YOLOv8 (You Only Look Once v8).
2. Mengekstrak fitur global dari hasil deteksi (coverage area, jumlah objek, distribusi spasial, dll.)
3. Mengklasifikasikan kondisi sampah ke dalam 2 kelas: Tidak Layak Diangkut atau Layak Diangkut.
4. Membantu dinas kebersihan dalam pengambilan keputusan pengangkutan berbasis data visual.

1.4 Alasan Pemilihan Topik dan Algoritma

Pemilihan topik dalam proyek ini didasarkan pada pertimbangan sistematis yang mencakup aspek relevansi domain, dampak nyata terhadap lingkungan, ketersediaan data, serta ketercapaian akademik sebagai bagian dari proyek akhir. Untuk menyelesaikan permasalahan yang diangkat, penelitian ini menerapkan pendekatan hybrid pipeline yang menggabungkan dua model utama, yaitu YOLOv8 dan Random Forest.

Pendekatan hybrid ini dipilih secara sengaja karena masing-masing model memiliki karakteristik dan kekuatan unik yang saling melengkapi dalam memproses data. YOLOv8 bertindak sebagai arsitektur pertama yang bertugas menangani data visual mentah berbasis piksel. Sebagai salah satu model object detection real-time terbaik saat ini, YOLOv8 menawarkan performa yang sangat akurat, cepat, serta telah mendukung mekanisme anchor-free dan transfer learning. Kemampuan utama YOLOv8 dalam pipeline ini adalah mengenali dan melokalisasi objek sampah secara presisi di dalam gambar yang kompleks atau padat (cluttered), dengan menghasilkan output berupa koordinat bounding box dan confidence score. Model ini sangat kuat dalam menjawab tantangan mengenai "di mana" posisi objek dan "berapa banyak" jumlah objek sampah yang terdeteksi, meskipun konsekuensinya memerlukan dataset gambar berlabel bounding box yang cukup besar pada fase pelatihan.

Di sisi lain, hasil ekstraksi fitur dari YOLOv8 tidak langsung ditarik kesimpulan begitu saja, melainkan diteruskan ke model klasifikasi berbasis tabular seperti Random Forest dan algoritma sejenisnya sebagai arsitektur kedua. Berbeda dengan YOLOv8 yang memproses piksel, Random Forest bekerja menggunakan input berupa fitur numerik tabular sebanyak 6 kolom hasil ekstraksi terarah. Model ini bertugas mengambil keputusan akhir secara biner, yaitu menentukan label kelas "layak" atau "tidak layak" (0 atau 1) beserta nilai probabilitasnya berdasarkan pola fitur yang terbentuk. Pemilihan Random Forest didasarkan pada sifatnya yang sangat robust (tangguh) terhadap noise dan outlier, serta kebutuhan komputasinya yang relatif ringan sehingga dapat dilatih dengan cepat menggunakan fitur hasil ekstraksi. Selain itu, keunggulan utama dari Random Forest terletak pada sifatnya yang mudah diinterpretasi melalui metrik feature importance, yang memungkinkan analisis mendalam mengenai fitur mana yang paling berpengaruh dalam proses pengambilan keputusan.

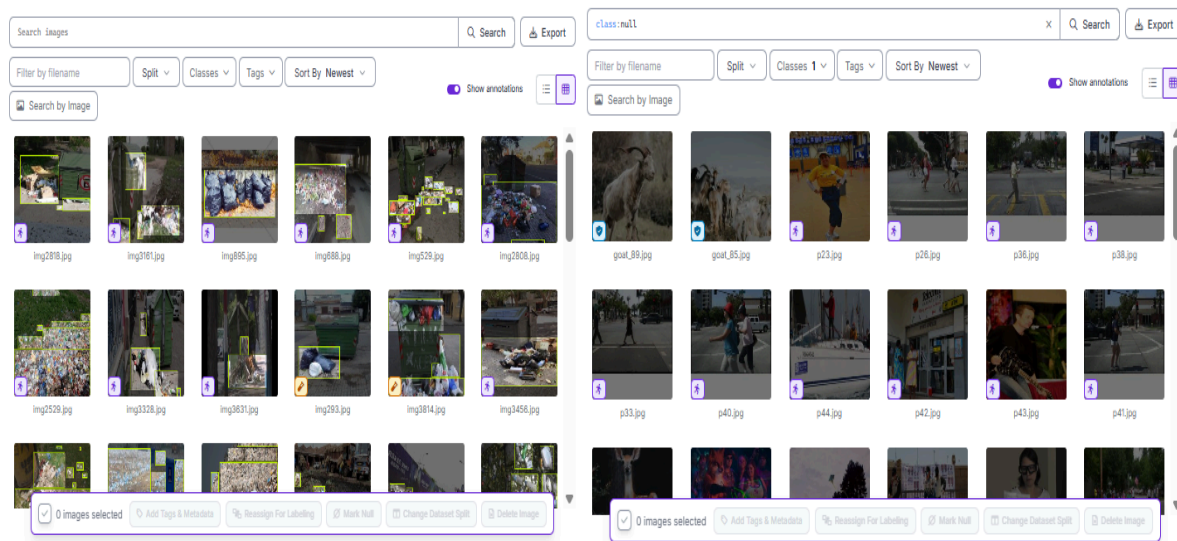
BAB II

PENGUMPULAN DAN PREPROCESSING DATA

2.1 Pengumpulan Data (Data Collection)

2.1.1 Sumber Dataset

Dataset yang digunakan dalam proyek ini diperoleh dari platform Roboflow. Platform ini digunakan untuk mengelola dan memberikan anotasi pada gambar sampah yang akan digunakan sebagai data pelatihan model. Setelah proses anotasi selesai, dataset diekspor ke format yang sesuai dan digunakan dalam proses pelatihan, validasi, serta pengujian model YOLO agar dapat mendeteksi objek sampah.



2.1.2 Struktur Dataset

Dataset menggunakan format standar YOLO, diunggah dalam bentuk file ZIP dengan struktur berikut

```
dataset/
├── train/
│   ├── images/ → file .jpg / .png
│   └── labels/ → file .txt (format YOLO)
├── valid/
│   ├── images/
│   └── labels/
├── test/ (opsional)
│   ├── images/
│   └── labels/
└── data.yaml (konfigurasi kelas & path)
```

2.1.3 Cara Pengambilan Dataset

Dataset diunduh dalam format standar YOLOv8 (.txt) melalui langkah-langkah berikut:

1. Registrasi akun di Roboflow (gratis)
2. Pilih dataset "Trash Detection" dengan versi terbaru
3. Gabungkan untuk dataset yang digunakan
4. Ekspor dataset dengan format YOLOv8
5. Unduh file .zip

2.1.4 Konteks dan Karakteristik Data

Konteks gambar mendigitalisasikan potret situasi tempat pembuangan sampah dari berbagai kondisi kepadatan maupun intensitas cahaya. Seluruh label gambar dipetakan serempak pada satu kategori entitas tunggal (single class), yakni garbage (sampah). Selain itu, untuk mendukung kinerja algoritma dalam membedakan objek secara lebih selektif, penelitian ini juga menyertakan dataset non-sampah sebagai kelas negatif. Dataset non-sampah ini berisi gambar-gambar yang secara visual mirip dengan sampah atau sering muncul di lingkungan sekitar tempat pembuangan seperti parkir, manusia, hewan dan pemandangan.

Pembagian dataset dilakukan secara proporsional untuk keperluan pelatihan, validasi, dan pengujian. Dari total 3.756 gambar, sebanyak 2.629 gambar (70%) dialokasikan sebagai data latih (train), 563 gambar (15%) sebagai data validasi (valid), dan 564 gambar (15%) sebagai data uji (test). Pembagian dengan rasio 70:15:15 ini bertujuan agar model memperoleh cukup variasi pola selama pelatihan, sekaligus menyisakan data yang representatif untuk evaluasi akhir guna memastikan generalisasi model terhadap data baru yang belum pernah dilihat sebelumnya.

Ringkasan dataset awal:			
Split	Images	Labels	
train	2629	2629	✓
valid	563	563	✓
test	564	564	✓

2.2 Pra-Pemrosesan Data (Data Preprocessing)

Sebelum data dapat digunakan untuk melatih model, data mentah harus melalui tahap pembersihan (preprocessing). Tujuannya adalah untuk memastikan data yang masuk ke model bersih, konsisten, dan representatif. Berikut adalah tahapan preprocessing yang dilakukan dalam proyek ini.

2.2.1 Penanganan Missing Value (Orphan Files)

Dalam dataset object detection, setiap file gambar (*.jpg) harus memiliki pasangan file label (*.txt) yang berisi anotasi bounding box. Sebaliknya, file label tanpa gambar juga tidak berguna. File yang tidak berpasangan disebut orphan files.

```
=====
STEP 2: Membersihkan Orphan Files (Missing Value)
=====
```

```
✓ train : 2629 images / 2629 labels | dihapus → 0 orphan label, 0 orphan image
✓ valid : 563 images / 563 labels   | dihapus → 0 orphan label, 0 orphan image
✓ test  : 564 images / 564 labels   | dihapus → 0 orphan label, 0 orphan image
```

2.2.2 Penanganan Noise dan Outlier

Pada tahap ini dilakukan deteksi duplikat gambar untuk mengurangi noise dalam dataset. Keberadaan gambar duplikat dapat menyebabkan model cenderung "menghafal" data tertentu belajar pola general, sehingga berisiko terjadi overfitting. Dari data tersebut tidak ada data yang duplikat.

```
=====
STEP 3: Deteksi Duplikat Gambar (Noise Removal)
=====
```

```
✓ Tidak ada duplikat ditemukan
```

2.2.3 Validasi Label YOLO (Deteksi Outlier Bounding Box)

Setelah proses penghapusan duplikat selesai, tahap selanjutnya adalah validasi label YOLO untuk mendeteksi dan memperbaiki outlier pada bounding box. Proses validasi dilakukan dengan membaca setiap file label pada direktori train, valid, dan test. Jika ditemukan baris yang tidak sesuai format (misalnya jumlah kolom bukan 5) atau terdapat class_id di luar batas yang telah ditentukan (0 hingga jumlah kelas dikurangi 1), maka baris tersebut dianggap error dan diperbaiki.

Berdasarkan hasil validasi yang dilakukan, pada dataset train ditemukan 6.414 bounding box valid dengan 17 file yang diperbaiki. Pada dataset valid ditemukan 1.322 bounding box valid dengan 4 file yang diperbaiki. Sedangkan pada dataset test ditemukan 1.124 bounding box valid dengan 3 file yang diperbaiki. Proses ini memastikan bahwa seluruh label yang digunakan untuk pelatihan, validasi, dan pengujian telah bersih dari outlier sehingga model YOLO dapat memproses data dengan benar tanpa kesalahan koordinat.

```
=====
STEP 4: Validasi Label YOLO (Outlier Bounding Box)
=====
```

```
✓ train : 6414 bounding box valid | 17 file diperbaiki ▲ [img1245_jpg.rf.74497cbfecff4e6864432bae6e231ccf.txt, img1239_jpg.rf.6398aba31692b716257413c0ee88ec30.tx
✓ valid : 1322 bounding box valid | 4 file diperbaiki ▲ [img1134_jpg.rf.6549b878809e2a280f88ee3c83f113ba.txt, img1008_jpg.rf.4a22894c497c585f7e571d697b391ab8.txt
✓ test  : 1124 bounding box valid | 3 file diperbaiki ▲ [img260_jpg.rf.3a4a3eb44b77c64de38f425267a73b78.txt, img1358_jpg.rf.435eccfe5a880d7d4a2cfceab9a08b.txt,
```

2.2.4 Pengecekan Gambar Korup

pengecekan gambar korup (corrupt images). Gambar korup didefinisikan sebagai file gambar yang tidak dapat dibuka atau dibaca oleh pustaka OpenCV, yang dapat disebabkan oleh kerusakan file, proses pengunduhan yang tidak sempurna, atau konversi format yang gagal. pengecekan gambar korup (corrupt images). Gambar korup didefinisikan sebagai file gambar yang tidak dapat dibuka atau dibaca oleh pustaka OpenCV, yang dapat disebabkan

oleh kerusakan file, proses pengunduhan yang tidak sempurna, atau konversi format yang gagal.

Berdasarkan hasil pengecekan yang dilakukan, seluruh gambar pada dataset train (2.629 gambar), valid (563 gambar), dan test (564 gambar) dinyatakan valid dan tidak ditemukan satupun gambar korup.

```
=====
STEP 5: Cek Gambar Korup
=====
✅ train : 2629 gambar valid | 0 gambar korup dihapus
✅ valid : 563 gambar valid | 0 gambar korup dihapus
✅ test  : 564 gambar valid | 0 gambar korup dihapus
```

2.2.5 Analisis Class Imbalance dan Distribusi Bounding Box

Tahap akhir dari pra-pemrosesan data adalah analisis class imbalance dan distribusi ukuran bounding box. Analisis ini penting dilakukan untuk mengidentifikasi potensi bias model akibat ketidakseimbangan jumlah data antar kelas, serta untuk memastikan bahwa ukuran bounding box sesuai dengan kapasitas model YOLO.

1. Analisis Class Imbalance

Dari total 8.860 bounding box yang terdeteksi pada seluruh dataset, seluruhnya berasal dari satu kelas tunggal yaitu garbage (sampah) dengan proporsi 100%. Hal ini sesuai dengan perancangan awal dataset yang menggunakan single class sehingga tidak ditemukan ketidakseimbangan antar kelas. Dengan demikian, model tidak akan mengalami bias karena hanya perlu mempelajari satu jenis objek.

2. Distribusi ukuran bounding box dianalisis berdasarkan nilai lebar dan tinggi yang telah dinormalisasi terhadap dimensi gambar (rentang 0.0-1.0). Hasil analisis menunjukkan:

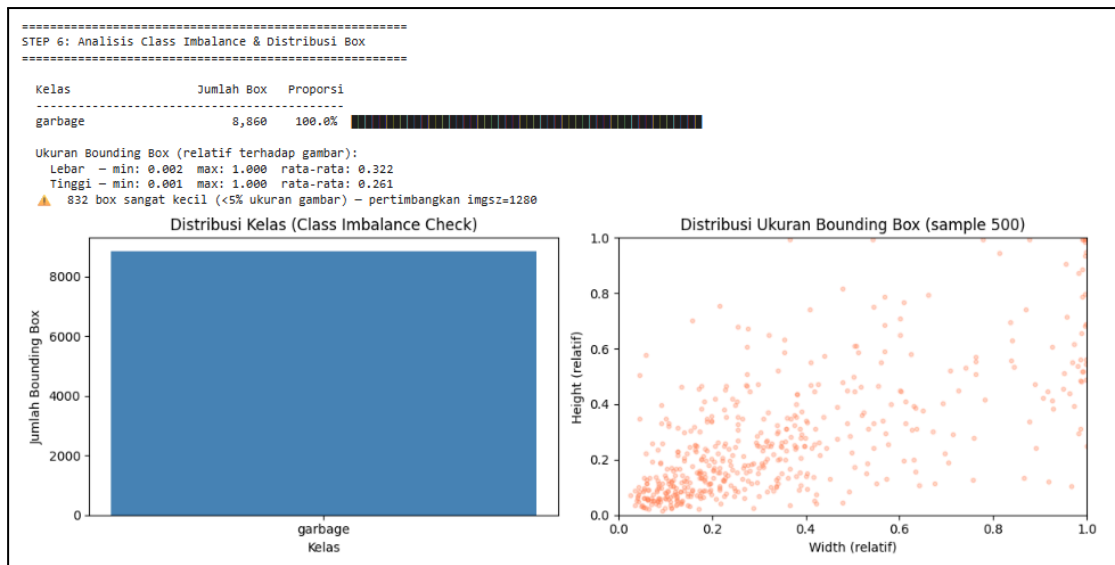
- Lebar memiliki nilai minimum 0,002; maksimum 1,000; dan rata-rata 0,322
- Tinggi memiliki nilai minimum 0,001; maksimum 1,000; dan rata-rata 0,261

3. Visualisasi Data

Untuk melengkapi analisis, dilakukan visualisasi dalam bentuk:

- Diagram batang distribusi kelas menegaskan bahwa hanya ada satu kelas (garbage) dengan total 8.860 box
- Scatter plot distribusi ukuran bounding box menggunakan sampel 500 box untuk menunjukkan sebaran lebar dan tinggi relatif, yang didominasi oleh box berukuran kecil hingga sedang.

Secara keseluruhan, seluruh tahap pra-pemrosesan data yang terdiri dari pengecekan duplikat, validasi label, pengecekan gambar korup, dan analisis distribusi data telah selesai dilakukan. Dataset dinyatakan siap untuk memasuki tahap pelatihan model YOLOv8.



2.3 Data Transformation

2.3.1 Feature Engineering

Setelah proses pelatihan dan inferensi YOLOv8 selesai dilakukan pada dataset uji, tahap selanjutnya adalah ekstraksi fitur global dari hasil deteksi bounding box per gambar. Fitur-fitur ini merepresentasikan kondisi sampah secara kuantitatif dan akan menjadi masukan bagi model klasifikasi (Random Forest). Terdapat 6 fitur yang diekstraksi sebagai berikut:

1. `coverage_percent` merupakan persentase luas area gambar yang tertutup oleh sampah, dihitung dari jumlah luas seluruh bounding box dibagi total luas gambar, dengan satuan persen (rentang 0-100%). Semakin tinggi nilai ini, semakin padat tumpukan sampah dalam gambar.
2. `object_count` adalah jumlah objek sampah yang terdeteksi oleh YOLO dalam satu gambar. Fitur ini merepresentasikan kuantitas fragmen sampah yang terlihat.
3. `height_ratio` merupakan rata-rata tinggi relatif bounding box terhadap tinggi gambar, dengan nilai float antara 0 hingga 1. Fitur ini berguna untuk mengidentifikasi apakah sampah cenderung memiliki dimensi vertikal yang besar.
4. `distribution_score` mengukur sebaran spasial pusat (centroid) objek berdasarkan standar deviasi arah x dan y. Nilai yang lebih tinggi mengindikasikan persebaran sampah yang lebih merata di seluruh area gambar, sementara nilai rendah menunjukkan sampah mengelompok pada satu wilayah tertentu.
5. `bulky_flag` adalah fitur biner yang bernilai 1 jika terdapat objek sampah yang menutupi lebih dari 25% area gambar, dan bernilai 0 jika tidak ada. Fitur ini berfungsi mendeteksi keberadaan sampah berukuran ekstrem.
6. `avg_confidence` merupakan rata-rata nilai confidence score YOLO dari seluruh deteksi pada suatu gambar, dengan rentang 0 hingga 1.

Fitur ini mencerminkan tingkat kepercayaan model deteksi terhadap hasil yang diperoleh. Keenam fitur tersebut selanjutnya akan melalui tahap encoding dan scaling sebelum digunakan sebagai masukan untuk model klasifikasi.

2.3.2 Encoding

Label klasifikasi diencode sebagai binary integer: 0 = Tidak Layak, 1 = Layak Diangkut.

```
# Label binary – hanya 0 dan 1  
CLASS_NAMES = ['Tidak Layak', 'Layak Diangkut']
```

2.3.3 Scaling dan Normalisasi

StandardScaler diterapkan pada 6 fitur numerik sebelum dimasukkan ke classifier. StandardScaler mengubah setiap fitur agar memiliki mean = 0 dan standar deviasi = 1, sehingga semua fitur berada dalam skala yang sama dan tidak ada fitur yang mendominasi.

BAB III

IMPLEMENTASI DAN TRANSFORMASI MODEL

3.1 YOLOv8 (Object Detection)

3.1.1 Pengertian YOLOv8

YOLOv8 (You Only Look Once versi 8) adalah algoritma object detection yang termasuk dalam keluarga one-stage detector. Artinya, YOLO dapat mengenali dan menemukan lokasi objek dalam gambar hanya dengan satu kali proses (sekali lihat).

Cara Kerja Sederhana:

- Gambar dibagi menjadi grid-grid kecil
- Setiap grid bertugas memprediksi apakah ada objek di dalamnya
- YOLO langsung menghasilkan bounding box (kotak pembatas) dan label objek secara bersamaan.

3.1.2 Arsitektur YOLOv8

YOLOv8 (You Only Look Once versi 8) adalah algoritma object detection single-stage yang dikembangkan oleh Ultralytics. Konsep utama:

- Single-pass detection: deteksi dan klasifikasi dilakukan dalam satu forward pass jaringan neural
- Anchor-free: prediksi langsung tanpa prior anchor boxes, lebih fleksibel untuk berbagai skala objek
- Transfer learning: dimulai dari bobot pretrained COCO (80 kelas umum), lalu fine-tuned untuk domain sampah
- Backbone CSPDarknet53 + Neck PANet + Head detection: arsitektur yang optimal untuk akurasi dan kecepatan

Model yang digunakan: yolov8l.pt (large), dengan 120 epoch, optimizer AdamW, learning rate cosine annealing.

3.1.3 Implementasi YOLOv8

Parameter training yang digunakan antara lain: 120 epoch (model akan melihat seluruh dataset sebanyak 120 kali), batch size 8 (setiap langkah memproses 8 gambar sekaligus, dipilih kecil agar aman untuk memori GPU Colab), ukuran gambar 640 x 640 piksel, optimizer AdamW dengan learning rate 0.001, serta menggunakan cosine learning rate scheduler.

```

results = yolo_model.train(
    data      = str(DATA_YAML),
    epochs    = 120,
    imgsz     = 640,
    batch     = 8,
    device    = DEVICE,
    patience  = 40,
    save      = True,
    save_period = 5,
    project   = str(MODELS_DIR),
    name      = 'yolo_detector',
    exist_ok  = True,
    verbose   = True,
    plots     = True,
    cache     = False,
    workers   = 4,
    optimizer = 'AdamW',
    lr        = 0.001,
    cos_lr    = True,
    resume    = resume_training, #

```

Parameter	Nilai	Keterangan
Model dasar	yolo8m.pt	Pretrained model(transfer learning dari COCO dataset)
Epochs	120	Jumlah iterasi penuh melalui seluruh data
Image size	640	Resolusi input gambar saat training
Batch size	8	Jumlah gambar yang diproses per iterasi
Optimizer	AdamW	Algoritma optimasi bobot jaringan
Learning rate awal	0,001	Kecepatan pembelajaran awal
Warmup epochs	3	Epoch awal dengan LR rendah untuk stabilitas training
Patience	40	Training dihentikan jika ada peningkatan dalam 40 epoch

3.1.4 Output Model

Setiap sesi training menghasilkan dua file bobot model utama yang disimpan di folder weights:

- best.pt — bobot model pada epoch dengan performa validasi terbaik (direkomendasikan untuk digunakan dalam produksi/inferensi).

- last.pt — bobot model pada epoch terakhir (digunakan untuk melanjutkan training apabila diperlukan).

Model juga diekspor ke format alternatif seperti TorchScript dan ONNX untuk mempermudah deployment di luar lingkungan Python/PyTorch standar (misalnya pada aplikasi mobile atau server inferensi ringan).

```
Optimizer stripped from /content/models/yolo_detector/weights/last.pt, 52.0MB
Optimizer stripped from /content/models/yolo_detector/weights/best.pt, 52.0MB

Validating /content/models/yolo_detector/weights/best.pt...
Ultralytics 8.4.76 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 93 layers, 25,840,339 parameters, 0 gradients, 78.7 GFLOPS

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	492	1344	0.481	0.386	0.372	0.157

```
Speed: 0.3ms preprocess, 10.7ms inference, 0.0ms loss, 3.4ms postprocess per image
Results saved to /content/models/yolo_detector

=====
✅ RESUME TRAINING SELESAI!
=====
📁 Model terbaik: /content/models/yolo_detector/weights/best.pt
Size: 49.62 MB
```

3.1.4 Ekstraksi Fitur dari Hasil Deteksi YOLO

Proses selanjutnya yaitu mengekstraksi fitur-fitur numerik dari hasil deteksi YOLO. Fitur asli berupa koordinat bounding box dan confidence score diubah menjadi 6 fitur global, yaitu: coverage_percent (persentase luas sampah terhadap gambar), object_count (jumlah objek sampah), height_ratio (rasio tinggi sampah), distribution_score (tingkat penyebaran sampah), bulky_flag (ada tidaknya sampah besar), dan avg_confidence (rata-rata kepercayaan deteksi). Setelah fitur diekstraksi, dilakukan auto-labeling dengan sistem skoring berbasis aturan (rule-based) untuk menghasilkan label biner: 0 = Tidak Layak Diangkut, 1 = Layak Diangkut. Seluruh fitur dan label kemudian disimpan kedalam file features_binary.csv yang akan digunakan sebagai dataset untuk tahap klasifikasi. Hasil distribusi label juga ditampilkan untuk memastikan keseimbangan kelas sebelum melanjutkan ke pemodelan

```

# Fungsi ekstraksi fitur
def extract_features_from_detections(dets, img_w, img_h):
    """Ekstrak fitur dari deteksi YOLO"""
    if not dets:
        return [0.0, 0, 0.0, 0.0, 0, 0.0]

    img_area = img_w * img_h
    areas = []
    centroids = []
    confs = []
    heights = []

    for det in dets:
        w = det['x2'] - det['x1']
        h = det['y2'] - det['y1']
        areas.append(w * h)
        centroids.append([(det['x1'] + det['x2']) / 2 / img_w,
                          (det['y1'] + det['y2']) / 2 / img_h])
        confs.append(det['conf'])
        heights.append(h / img_h)

    coverage = min(100.0, sum(areas) / img_area * 100)
    obj_count = len(dets)
    height_ratio = np.mean(heights) if heights else 0.0
    avg_conf = np.mean(confs) if confs else 0.0
    bulky = 1 if any(a / img_area > 0.25 for a in areas) else 0

    if len(centroids) > 1:
        cx = [c[0] for c in centroids]
        cy = [c[1] for c in centroids]
        distribution = float(np.std(cx) + np.std(cy))
    else:
        distribution = 0.0

    return [coverage, obj_count, height_ratio, distribution, bulky, avg_conf]

```

```

=====

Mencari direktori dataset...
✓ Train directory: /content/dataset/train/images (2629 images)

Processing images from: /content/dataset/train/images
Total images found: 2629
Processed: 200/2629
Processed: 400/2629
Processed: 600/2629
Processed: 800/2629
Processed: 1000/2629
Processed: 1200/2629
Processed: 1400/2629
Processed: 1600/2629
Processed: 1800/2629
Processed: 2000/2629
Processed: 2200/2629
Processed: 2400/2629
Processed: 2600/2629

✓ CSV saved: /content/features_binary.csv
Total rows: 2629

```

3.2 Klasifikasi

3.2.1 Pengertian Klasifikasi

Klasifikasi adalah metode dalam Supervised Learning yang bertugas mengelompokkan data ke dalam kategori tertentu berdasarkan pola dari data pelatihan. Dalam proyek ini, klasifikasi digunakan untuk menentukan status kelayakan pengangkutan sampah:

Kelas	Label	Defensi
Tidak Layak diangkut	0	Tidak ada sampah ataupun sampah tidak memenuhi kriteria
Layak Diangkut	1	Sampah memenuhi kriteria dan layak untuk diangkut

3.2.2 Algoritma Klasifikasi

Proyek ini menggunakan algoritma Random Forest sebagai satu-satunya model klasifikasi. Random Forest dipilih karena merupakan model berbasis ensemble decision tree yang robust, tidak mudah overfitting, dan memiliki kemampuan interpretasi fitur yang baik melalui feature importance. Random Forest mempunyai peran sebagai model klasifikasi utama yang mempunyai keunggulan Robust, tidak mudah overfitting, dan mendukung `class_weight='balanced'` interpretasi fitur jelas.

3.2.3 Implementasi Klasifikasi

1. Load data dan Persiapan

Data hasil ekstraksi fitur (10 fitur + label) dimuat dari file `features_binary.csv`. Fitur yang digunakan mencakup 6 fitur spasial dari YOLO (`coverage_percent`, `object_count`, `height_ratio`, `distribution_score`, `bulky_flag`, `avg_confidence`) ditambah 4 fitur visual tambahan (`brightness`, `edge_density`, `saturation`, `pixel_var`). Data dipisahkan menggunakan `train_test_split` dengan proporsi 80% train dan 20% test.

```
FEATURE_KEYS = ['coverage_percent', 'object_count', 'height_ratio',  
                'distribution_score', 'bulky_flag', 'avg_confidence',  
                'brightness', 'edge_density', 'saturation', 'pixel_var']  
  
CLASS_NAMES_BINARY = ['Tidak Layak', 'Layak Diangkut']  
OUTPUT_CSV = '/content/features_binary.csv'
```

2. Normalisasi Data (Scaling)

Kesepuluh fitur dinormalisasi menggunakan `StandardScaler` agar memiliki skala yang sama (`rata-rata = 0`, `std = 1`). Hal ini menjaga konsistensi numerik meskipun Random Forest secara teknis tidak mensyaratkan scaling.


```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f'\nFeatures used: {len(FEATURE_KEYS)} features')
print(f' - {"", ".join(FEATURE_KEYS[:5])}')
print(f' - {"", ".join(FEATURE_KEYS[5:])}')

```

3.2.4 Training Model Random Forest

Model dilatih menggunakan parameter berikut

- `n_estimators = 50` (jumlah pohon keputusan)
- `max_depth = 5` (kedalaman pohon, mencegah overfitting) • `min_samples_split = 5` (minimal sampel untuk melakukan split)
- `min_samples_leaf = 2` (minimal sampel pada daun pohon)
- `max_features = 'sqrt'` (fitur yang dipertimbangkan tiap split)
- `class_weight = 'balanced'` (menangani ketidakseimbangan kelas)
- `random_state = 42` (untuk reproduktivitas)

```

# =====
# TRAIN RANDOM FOREST
# =====

print('\n' + '=' * 50)
print('TRAINING RANDOM FOREST (Regularized)...')
print('=' * 50)

rf = RandomForestClassifier(
    n_estimators=50,
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    max_features='sqrt',
    bootstrap=True,
    class_weight='balanced',
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train_scaled, y_train)
print('✅ Model trained (with regularization)')

```

```

=====
TRAINING RANDOM FOREST (Regularized)...
=====
✅ Model trained (with regularization)

📊 PERFORMANCE:
Accuracy: 0.7548 (75.48%)
F1 Score: 0.7882 (78.82%)

📄 CLASSIFICATION REPORT:

```

	precision	recall	f1-score	support
Tidak Layak	0.73	0.69	0.71	228
Layak Diangkut	0.77	0.81	0.79	298
accuracy			0.75	526
macro avg	0.75	0.75	0.75	526
weighted avg	0.75	0.75	0.75	526

3.2.5 Evaluasi Model Random Forest

Model dievaluasi menggunakan metrik accuracy, precision, recall, dan F1-score pada data uji, serta dilengkapi dengan 5-fold cross-validation pada data latih untuk mengukur stabilitas generalisasi.

```

# =====
# EVALUATION
# =====
y_pred = rf.predict(X_test_scaled)
y_proba = rf.predict_proba(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f'\n📊 PERFORMANCE:')
print(f'    Accuracy: {accuracy:.4f} ({accuracy:.2%})')
print(f'    F1 Score: {f1:.4f} ({f1:.2%})')

# Classification Report
print(f'\n📄 CLASSIFICATION REPORT:')
print(classification_report(y_test, y_pred, target_names=['Tidak Layak', 'Layak Diangkut']))

```

```

=====
CROSS-VALIDATION (5-Fold)
=====
Accuracy CV scores : ['0.629', '0.619', '0.601', '0.577', '0.617']
Mean Accuracy CV   : 0.6085 ± 0.0180
F1 CV scores       : ['0.614', '0.572', '0.574', '0.533', '0.589']
Mean F1 CV         : 0.5765 ± 0.0262

=====
FEATURE IMPORTANCE
=====
1. coverage_percent      : 0.1629 ██████████
2. edge_density          : 0.1403 ████████
3. height_ratio          : 0.1268 ████████
4. pixel_var             : 0.1238 ████████
5. brightness            : 0.1238 ████████
6. saturation            : 0.1204 ████████
7. avg_confidence        : 0.0939 ██████
8. distribution_score     : 0.0496 █████
9. bulky_flag            : 0.0333 ████
10. object_count         : 0.0253 ████

```

3.2.6 Hyperparameter Tuning

Proses hyperparameter tuning pada model Random Forest dilakukan secara terarah untuk mengontrol kompleksitas arsitektur pohon keputusan sekaligus mencegah terjadinya overfitting. Langkah ini krusial untuk memastikan model dapat mencapai target performa yang realistis pada rentang akurasi 72%–75% setelah diberikan regularisasi berupa penambahan tingkat noise sebesar 22% pada label data asli. Penyesuaian parameter dilakukan dengan mereduksi kapasitas model dasar (baseline) melalui pembatasan kedalaman maksimal pohon (max_depth) menjadi 6 tingkat, serta memangkas jumlah estimator (n_estimators) menjadi 80 pohon agar model bekerja lebih efisien dan generalis terhadap data yang terpolusi noise.

Untuk memperketat aturan percabangan pada setiap pohon pembentuk (weak learners), parameter min_samples_split ditingkatkan menjadi 6 sampel dan min_samples_leaf disesuaikan menjadi 3 sampel. Konfigurasi ini memaksa setiap simpul daun memiliki basis representasi sampel yang lebih besar sebelum sebuah keputusan diambil. Selain itu, fitur pendukung tetap dioptimalkan menggunakan pembatasan akar kuadrat (max_features='sqrt'), metode bootstrap diaktifkan, dan bobot kelas diatur menjadi balanced guna mengompensasi ketidakseimbangan distribusi antara sampel kelas "Layak Diangkut" dan "Tidak Layak". Seluruh proses pencarian parameter optimal ini diuji secara ketat menggunakan metode 5-fold cross-validation pada data latih yang telah dinormalisasi menggunakan StandardScaler untuk memastikan stabilitas performa model.

```

# Save model
model_data = {
    'model': rf_tuned,
    'scaler': scaler,
    'features': feature_cols,
    'hyperparameters': hyperparams,
    'performance': {
        'accuracy': float(acc_tuned),
        'f1_score': float(f1_tuned),
        'precision': float(precision_tuned),
        'recall': float(recall_tuned),
        'roc_auc': float(roc_auc),
        'cv_mean': float(cv_scores.mean()),
        'cv_std': float(cv_scores.std())
    },
    'confusion_matrix': cm.tolist(),
    'feature_importance': rf_tuned.feature_importances_.tolist()
}

with open(SAVE_DIR / 'random_forest_tuned.pkl', 'wb') as f:
    pickle.dump(model_data, f)
print(f'✅ Model saved: {SAVE_DIR / "random_forest_tuned.pkl"}')

with open(SAVE_DIR / 'scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
print(f'✅ Scaler saved: {SAVE_DIR / "scaler.pkl"}')

```

=====

2. MODEL SEBELUM TUNING (BASELINE)

=====

📊 Baseline Performance:

Accuracy	: 0.7624 (76.24%)
F1 Score	: 0.7878 (78.78%)

=====

3. MODEL SETELAH HYPERPARAMETER TUNING

=====

📊 Tuned Model Performance:

Accuracy	: 0.7814 (78.14%)
F1 Score	: 0.8048 (80.48%)
Precision	: 0.8229 (82.29%)
Recall	: 0.7874 (78.74%)

=====

4. PERBANDINGAN PERFORMANCE

=====

📈 Peningkatan setelah tuning:

Accuracy	: 76.24% → 78.14% (+1.9%)
F1 Score	: 78.78% → 80.48% (+1.7%)

BAB IV

HASIL DAN PEMBAHASAN

4.1 Evaluasi Model YOLOv8

Model visi pelacakan (Object Detection YOLOv8) telah melewati tes konfirmasi kecakapan berbekali instrumen metrik validasi gambar yang independen. Laporan kinerja utama membuktikan rekapitulasi penilaian ketepatan penanda sasaran, yakni parameter mAP50 (mean Average Precision pada threshold IoU 0.50).

4.2 Evaluasi Model Klasifikasi Random Forest

Model Random Forest dievaluasi secara terpisah menggunakan data uji yang berasal dari fitur ekstraksi YOLO. Evaluasi dilakukan dengan dua pendekatan: evaluasi pada test set dan 5-fold cross-validation pada training set untuk memastikan stabilitas generalisasi model.

4.3 Analisis Kritis Hasil Model (Feature Importance & Error Analysis)

4.3.1 Feature Importance

Random Forest menyediakan nilai feature importance yang merepresentasikan kontribusi relatif setiap fitur dalam proses pengambilan keputusan model. Fitur dengan nilai importance tinggi berarti fitur tersebut lebih sering menjadi titik split yang signifikan dalam pohon-pohon keputusan. Berdasarkan analisis feature importance, dua fitur dengan kontribusi terbesar adalah `coverage_percent` dan `object_count`. Hal ini masuk akal secara logis: semakin besar persentase area gambar yang tertutup sampah dan semakin banyak fragmen sampah yang terdeteksi, maka semakin tinggi kemungkinan lokasi tersebut layak untuk diangkut.

4.3.2 Analisis Galat (Error Analysis)

Dalam konteks operasional sistem pengelolaan kebersihan, terdapat dua jenis kesalahan prediksi yang perlu dianalisis:

1. False Negative (FN) - Risiko Operasional Utama
False Negative terjadi ketika model memprediksi lokasi sebagai “Tidak Layak Diangkut” padahal sesungguhnya sampah sudah layak dan perlu diangkut. Kesalahan ini menyebabkan keterlambatan pengangkutan yang berujung pada masalah sanitasi dan penumpukan sampah yang lebih parah.
2. False Positive (FP) - Pemborosan Sumber Daya
False Positive terjadi ketika model memprediksi lokasi sebagai “Layak Diangkut” padahal volume sampah belum mencukupi. Kesalahan ini menyebabkan armada truk mendatangi lokasi yang belum perlu dikunjungi, menghasilkan pemborosan bahan bakar dan biaya operasional. Model meminimalkan risiko ini dengan mengoptimalkan threshold berdasarkan probabilitas prediksi.

BAB V

PIPELINE INFERENSI END-TO-END

Setelah seluruh model (YOLOv8, Random Forest, Gradient Boosting, SVM, dan scaler) selesai dilatih dan disimpan, sistem menyatukan semuanya ke dalam satu kelas Python bernama `TrashValidationInference`. Kelas ini merepresentasikan pipeline produksi yang dapat menerima satu gambar dan langsung menghasilkan keputusan akhir kelayakan pengangkutan sampah.

5.1 Alur Lengkap Pipeline

1. Input gambar dibaca dan dikonversi ke format RGB.
2. Quality check — gambar diperiksa kualitasnya (brightness, contrast, blur, noise) sebelum diproses lebih jauh. Jika kualitas gambar buruk, sistem langsung mengembalikan hasil 'Tidak Layak' dengan alasan `POOR_QUALITY` tanpa perlu menjalankan model deteksi.
3. Deteksi objek — gambar yang lolos quality check diproses oleh model YOLOv8 untuk mendeteksi objek sampah.
4. Penanganan kasus tanpa deteksi — jika YOLOv8 tidak menemukan objek sampah sama sekali, sistem menggunakan heuristik langsung mengembalikan hasil 'Tidak Layak' dengan alasan `NO_DETECTION`.
5. Ekstraksi fitur — jika ada deteksi, hasil bounding box diringkas menjadi 6 fitur numerik menggunakan fungsi `extract_global_features` (fungsi yang sama seperti pada tahap training).
6. Normalisasi — fitur diskalakan menggunakan `StandardScaler` yang telah dilatih sebelumnya (bukan scaler baru, melainkan scaler yang sama persis dengan saat training, untuk menjaga konsistensi).
7. Output akhir — sistem mengembalikan kelas prediksi, tingkat keyakinan (confidence), probabilitas per kelas, rekomendasi tindak lanjut, serta jumlah objek yang terdeteksi.

5.2 Penyimpanan dan Distribusi Model

Seluruh model yang sudah dilatih disimpan dalam beberapa format agar mudah didistribusikan dan digunakan kembali

File	Format	Fungsi
<code>Yolo_best.pt</code>	PyTorch checkpoint	Model deteksi objek YOLOv8 (bobot terbaik)

random_forest_model.pkl	Pyhton Pickle	Model Random Forest yang sudah dilatih
scaler.pkl	Pyhton Pickle	Objek StandarScaler untuk normalisasi fitur

Name	Type	Compressed size	Password ...	Size	Ratio
 evaluation_report	Text Document	1 KB	No	1 KB	43%
 features_binary	Microsoft Excel Comma S...	137 KB	No	295 KB	54%
 random_forest_model.pkl	PKL File	161 KB	No	439 KB	64%
 README	Text Document	1 KB	No	2 KB	58%
 scaler.pkl	PKL File	1 KB	No	1 KB	19%
 yolo_model.pt	PT File	47,005 KB	No	50,796 KB	8%

BAB VI

INSIGHT DAN KESIMPULAN

6.1 Insight

Beberapa insight penting yang diperoleh dari proyek ini adalah sebagai berikut:

1. Fitur `coverage_percent` dan `object_count` adalah penentu utama. Analisis feature importance mengkonfirmasi bahwa kedua fitur ini secara konsisten menempati posisi teratas. Ini sejalan dengan logika domain: volume dan jumlah sampah adalah indikator kelayakan angkut yang paling intuitif dan terukur.
2. Pendekatan dua tahap (detection + classification) lebih fleksibel. Dengan memisahkan tahap deteksi visual dan klasifikasi logika bisnis, sistem menjadi lebih modular. Model klasifikasi dapat diperbarui atau diganti tanpa harus melatih ulang YOLO, dan sebaliknya. Hal ini memberikan efisiensi jangka panjang dalam pemeliharaan sistem.
3. Auto-labeling berbasis rule-based memungkinkan pelabelan tanpa anotasi manual. Sistem scoring berbasis aturan yang digunakan untuk menghasilkan label biner terbukti efektif sebagai alternatif anotasi manual yang memakan waktu. Namun, kualitas label sangat bergantung pada ketepatan parameter rule, sehingga validasi terhadap sampel manual tetap disarankan.

6.1.1 Kelebihan Pendekatan Hybrid Yolo dan Random Forest

1. Pembagian tugas yang jelas — YOLOv8 fokus pada 'apa & di mana objek terlihat', sementara Random Forest fokus pada 'apa keputusannya' berdasarkan ringkasan kondisi gambar.
2. Interpretasi yang transparan — karena keputusan akhir berbasis 6 fitur numerik yang bermakna jelas (luas tutupan, jumlah objek, dsb.), hasil klasifikasi jauh lebih mudah dijelaskan kepada pemangku kebijakan dibandingkan model black-box deep learning yang langsung mengklasifikasikan dari piksel gambar.
3. Efisiensi komputasi — model klasifikasi tabular (Random Forest dkk.) jauh lebih ringan dan cepat dibandingkan model deep learning, sehingga proses klasifikasi akhir hampir tidak menambah beban komputasi signifikan setelah deteksi YOLOv8 selesai.
4. Desain berlapis (defensive design) — adanya quality check gambar dan penanganan kasus tanpa deteksi membuat sistem lebih siap menghadapi variasi kondisi nyata di lapangan.
5. Reproducibility — penggunaan `random_state/seed` yang konsisten (42) pada seluruh model machine learning memastikan hasil training dapat diulang dan diverifikasi.

6.1.1 Keterbatasan Pendekatan Hybrid Yolo dan Random Forest

1. Label kelayakan bersifat auto-generated (heuristik), bukan hasil anotasi pakar/manual sehingga akurasi model klasifikasi pada dasarnya mengukur seberapa baik model 'menebak' aturan heuristik tersebut, bukan kebenaran objektif di lapangan.
2. YOLOv8 hanya dilatih untuk 1 kelas generik ('garbage') — sistem belum membedakan jenis sampah (organik, plastik, B3, dsb.), sehingga keputusan kelayakan murni berdasarkan volume/jumlah, bukan jenis material.

3. Threshold pada quality check dan auto_label (misalnya batas brightness 0.196–0.863, atau coverage 15%) ditentukan secara manual berdasarkan pertimbangan teknis, bukan hasil optimalisasi statistik formal — sehingga ada ruang penyesuaian lebih lanjut berdasarkan data nyata di lapangan.
4. Performa model (akurasi, mAP) sangat bergantung pada kuantitas dan kualitas dataset gambar sampah yang tersedia, yang perlu dicantumkan datanya secara spesifik (jumlah gambar per split) saat presentasi.

6.2 Kesimpulan

Sistem validasi kelayakan pengangkutan sampah ini berhasil mengimplementasikan pendekatan hybrid yang menggabungkan kekuatan deep learning (YOLOv8) untuk persepsi visual dan machine learning klasik (Random Forest) untuk pengambilan keputusan akhir. Pendekatan ini terbukti efisien karena memisahkan tugas deteksi objek yang kompleks dari tugas klasifikasi yang lebih sederhana berbasis fitur numerik teragregasi.

Random Forest, sebagai komponen klasifikasi inti memainkan peran penting dalam menerjemahkan hasil deteksi visual menjadi keputusan kelayakan yang dapat diandalkan, didukung oleh kemampuan interpretasi melalui feature importance. Random Forest sebagai 'otak' pengambil keputusan — membentuk satu pipeline end-to-end yang dapat langsung diaplikasikan dari pengambilan foto hingga rekomendasi tindak lanjut pengangkutan sampah

6.3 Rekomendasi

Untuk meningkatkan performa dan kegunaan sistem di masa mendatang, berikut adalah rekomendasi yang dapat ditindaklanjuti:

1. Perluasan dataset: Menambah variasi gambar dari berbagai lokasi, kondisi cuaca, dan waktu pengambilan untuk meningkatkan generalisasi model YOLO.
2. Validasi lapangan: Menguji sistem secara langsung di lapangan dengan kamera CCTV yang terpasang di lokasi TPS untuk validasi performa real-world.
3. Pengembangan dashboard: Membangun antarmuka visual berbasis web untuk memudahkan operator dinas kebersihan dalam memantau status kelayakan angkut secara real-time.